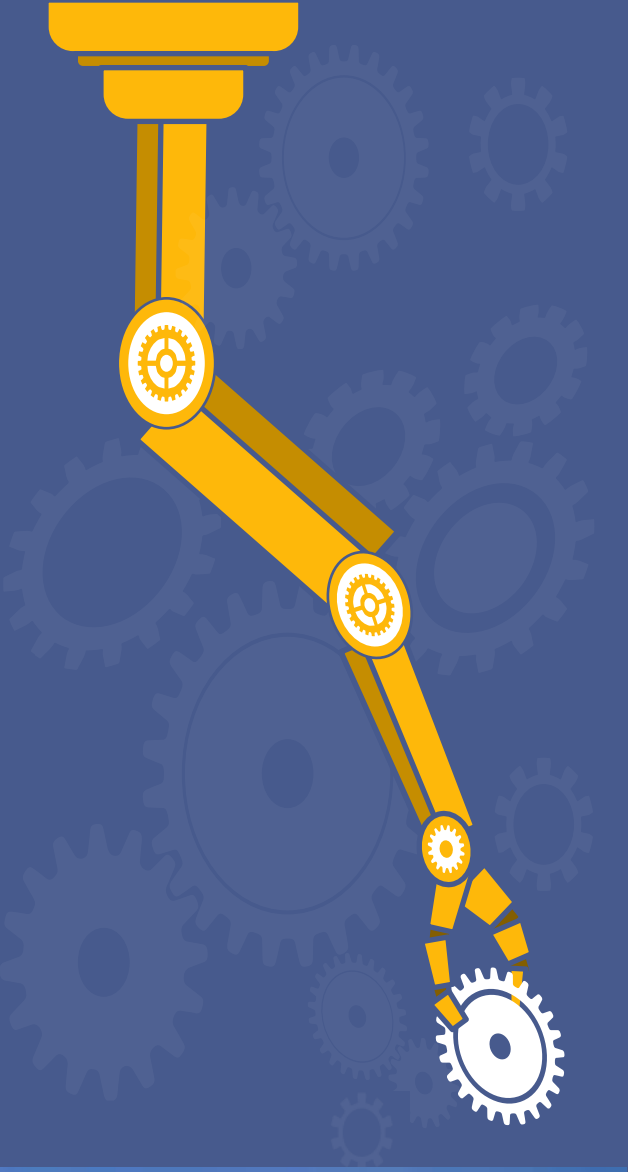


VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT, Vellore.

BCSE204L- Design and Analysis of Algorithms-

Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms

2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively

3. To synthesize efficient algorithms in various engineering design situations

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms- Course outcome

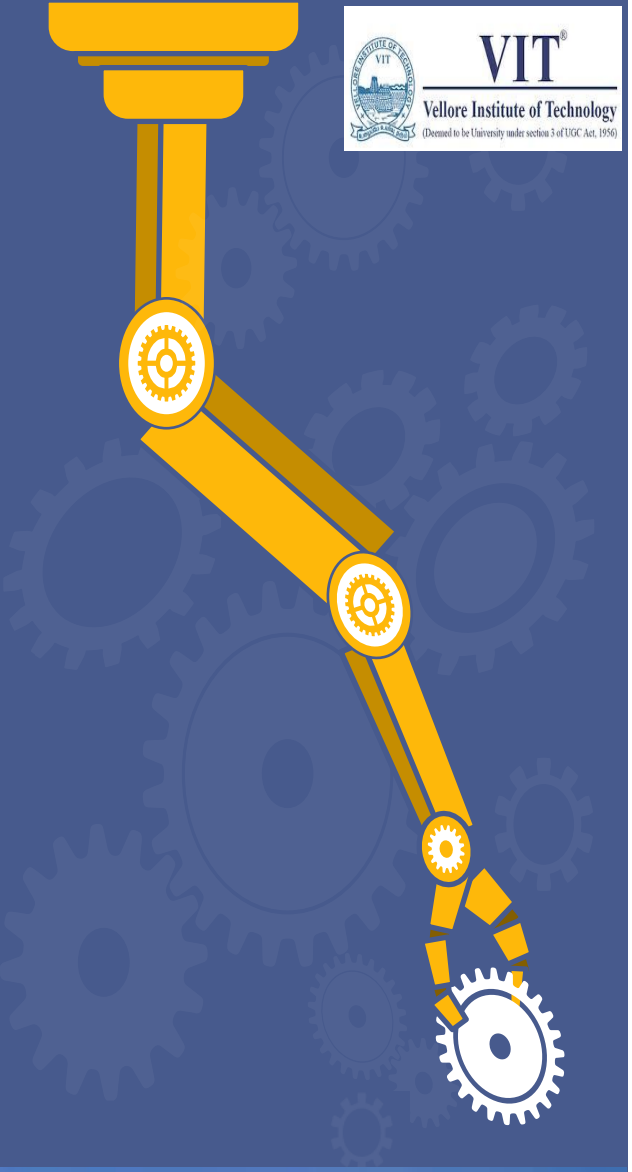
1. Apply the mathematical tools to analyse and derive the running time of the algorithms

2. Demonstrate the major algorithm design paradigms.

3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.

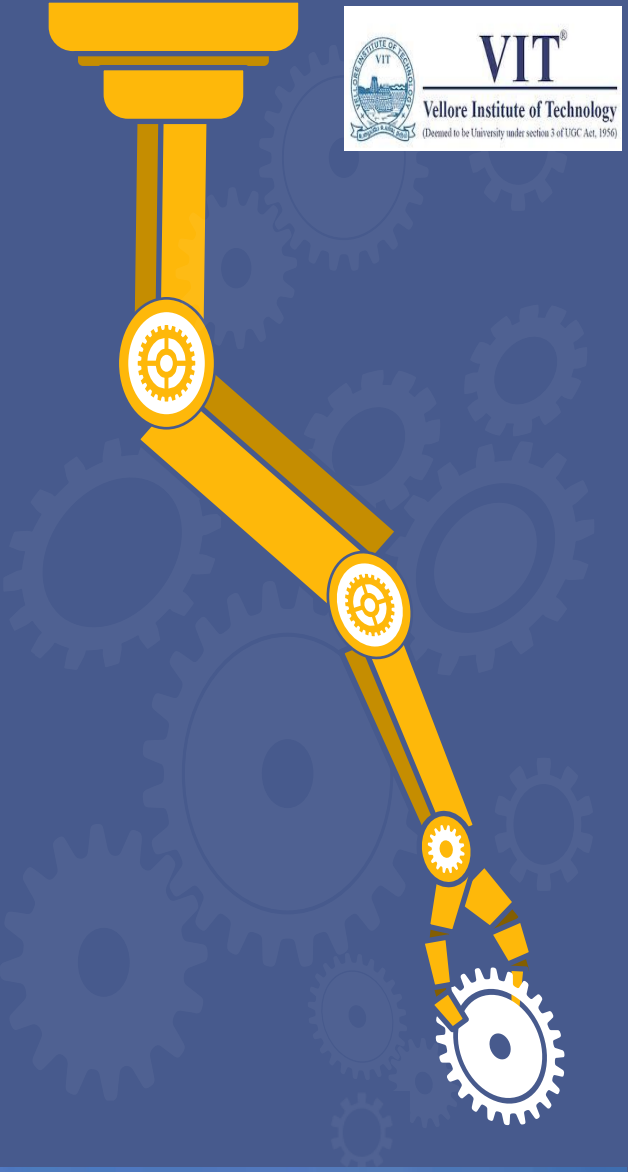
4. Articulating Randomized Algorithms.

5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	**Design Paradigms: Greedy, Divide and Conquer Techniques**	**6 hours**
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	**Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques**	**10 hours**
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	**String Matching Algorithms**	**5 hours**
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	**Graph Algorithms**	**6 hours**
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	**Geometric Algorithms**	**4 hours**
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	**Randomized algorithms**	**5 hours**
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	**Classes of Complexity and Approximation Algorithms**	**7 hours**
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	**Contemporary Issues**	**2 hours**



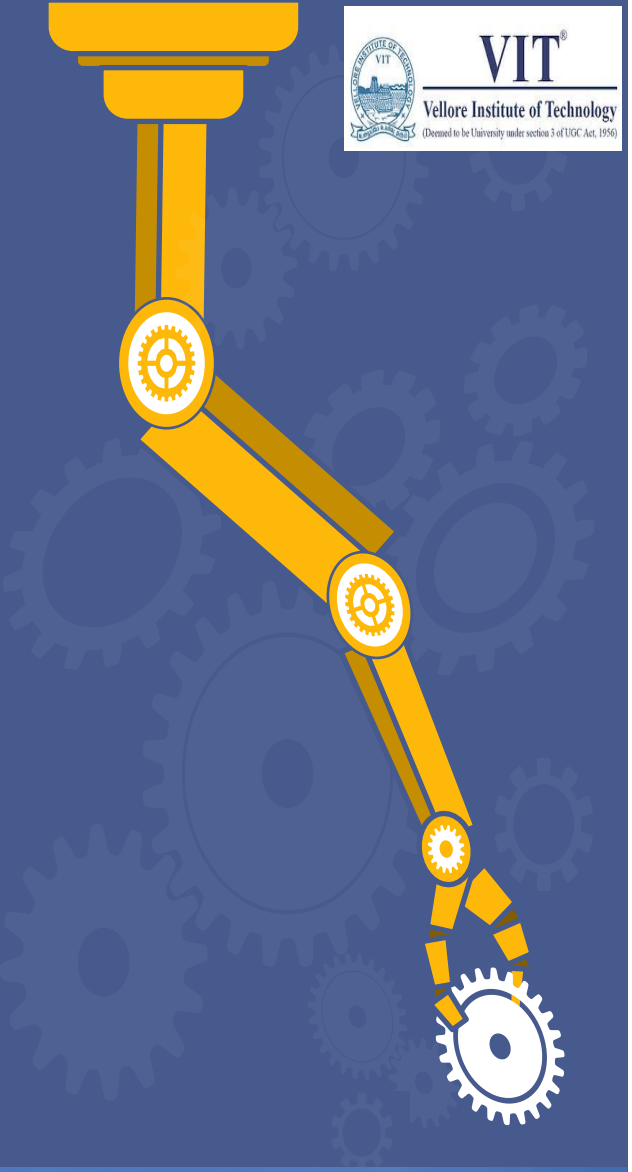
BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, ÉvaTardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press,1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B.Orlin, " Network Flows: Theory, Algorithms and Applications", Pearson Education, 2014.



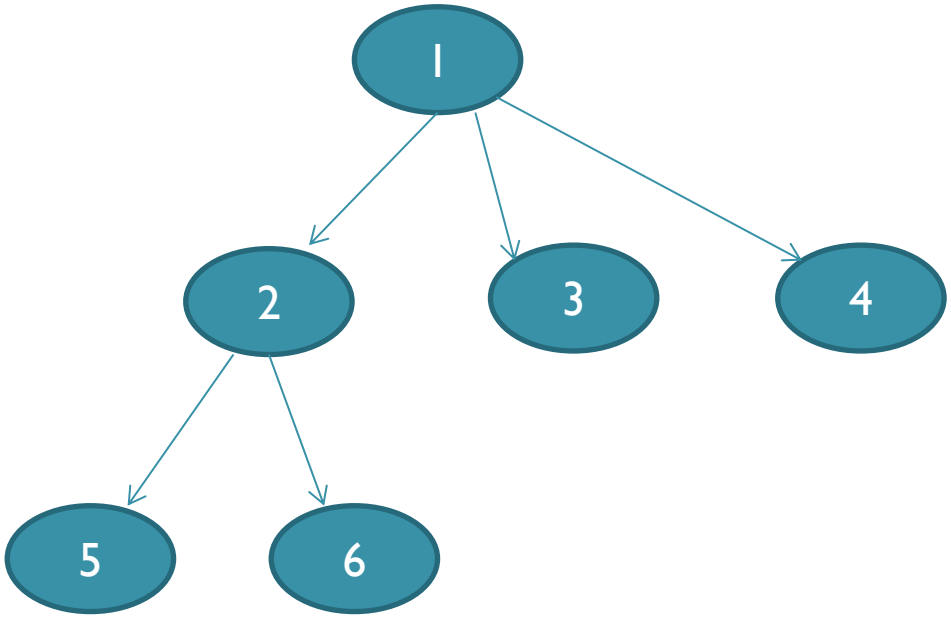
Module 2: Design Paradigms: Dynamic Programming-Backtracking- Branch and Bound(10 Hours)

- **Dynamic Programming**
  - Matrix chain Multiplication
  - Assembly line Scheduling
  - Longest Common Subsequence
  - 0/1 Knapsack Problem
  - TSP- Travelling Salesman Problem
- **Backtracking**
  - N-queens Problem
  - Sum of Subsets
  - Graph Coloring
- **Branch and Bound**
  - **LIFO and FIFO Methods**
  - **0/1 Knapsack Problem**
  - Job Selection Problem

Branch and Bound

- Another Problem Strategy
- Similar to Backtracking(Not Exactly) because it uses state space tree for solving the problem.
- Used for solving optimization problem.(Minimization Problem).
- It performs graph traversal on the state space tree using BFS instead of DFS.
  - FIFO(First in First Out)
  - LIFO(Last in First Out)
  - LCBB(Least Cost Branch and Bound)

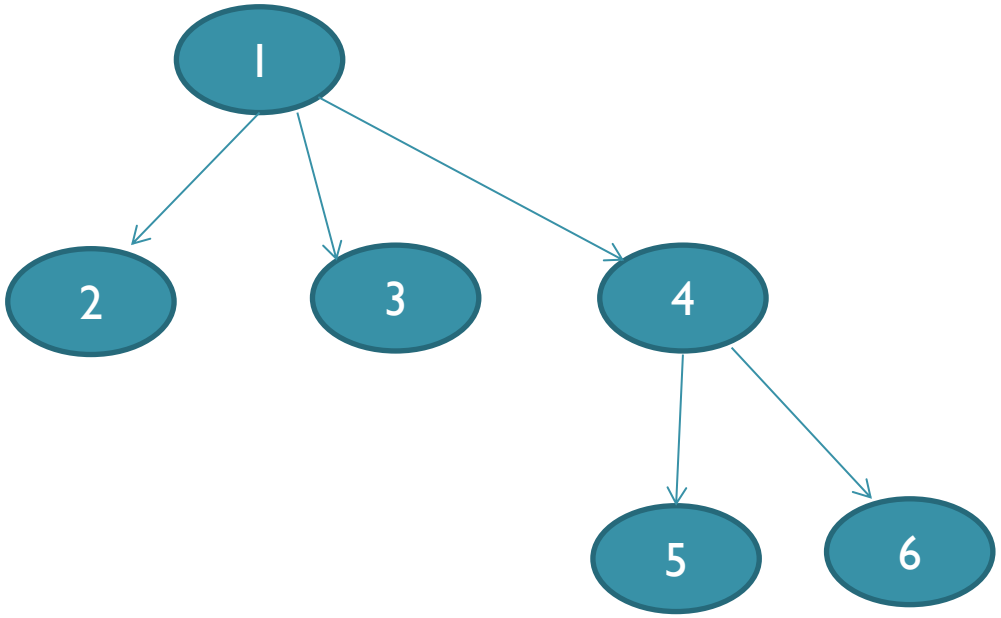
FIFO Branch and Bound



1	2	3	4	5	6

Queue – BFS
Order

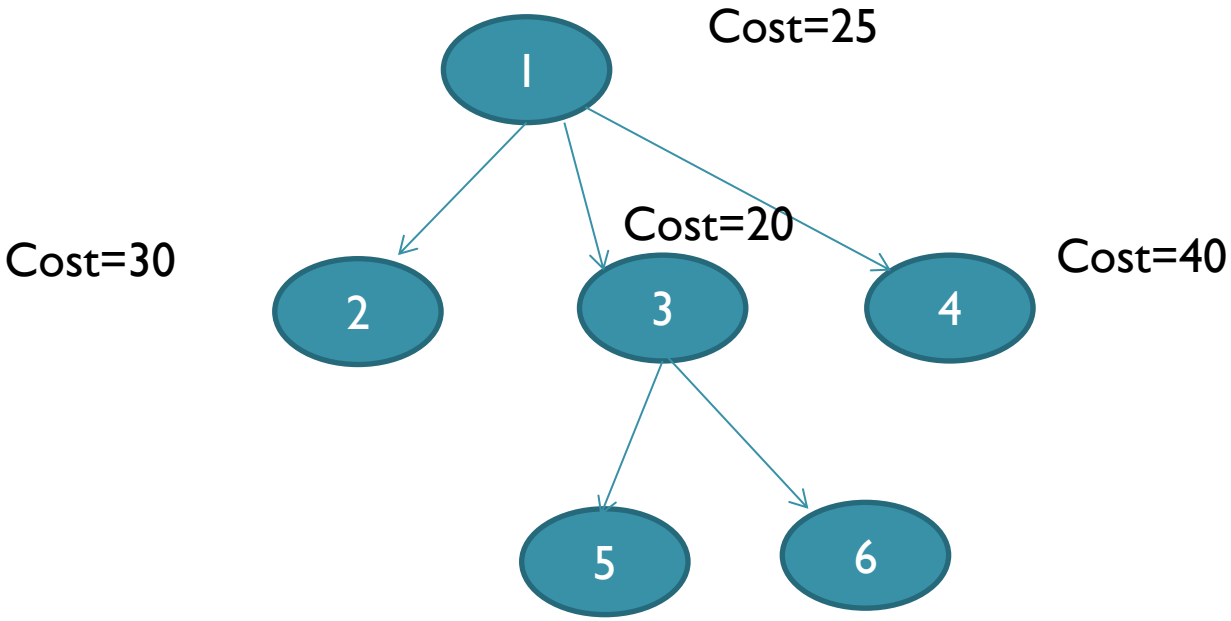
LIFO Branch and Bound



Stack– DFS Order

6
5
4
3
2
1

LCBB Branch and Bound



Expanding Least cost

LCBB SEARCH

- In both LIFO and FIFO Branch and Bound the selection rule for the next E-node is rigid and blind.
- The selection rule for the next E-node does not give any preference to a node that has a very good chance of getting the search to an answer node quickly.
- Search for an answer node can be speeded by using an **"intelligent" ranking function c(.)** for live nodes. The next E-node is selected on the basis of this ranking function.

LCBB SEARCH

- In LCBB search, we take the least one only and expand them by avoiding all others.
- The node x is assigned a rank using:

$$c(x) = f(x) + g(x)$$

- where, c(x) is the cost of x/ estimated minimum cost to reach the goal node
- f(x) is the cost of reaching x from the root/ number of moves from initial state.
- g(x) is an estimate of the additional effort needed to reach an answer node from x.

0/1 Knapsack using LCBB

- Here we do it for minimization- Here consider the profit as negative.
- **Formally the Problem can be Stated as:**

Minimize $\sum_{1 \le i \le n} p_i x_i$ → 1

subject to $\sum_{1 \le i \le n} w_i x_i \le m$ → 2

and Xi = 0 or 1 $1 \le i \le n$ → 3

0/1 Knapsack Problem – LCBB

Consider the following instance of the knapsack problem:

Number of objects(n) =5

Knapsack(capacity)= 12

(P_1,P_2,P_3,P_4,P_5)= (10,15,6,8,4) &

(w_1,w_2,w_3,w_4,w_5) = (4,6,3,4,2).

Procedure Attached in separate file.